Tutorial: Reading UML Class Diagrams (Using **Mermaid Code** as the Notation)

0) Key Idea (No Confusion Rule ☑)

Before we start, students must learn this distinction:

UML vs Mermaid

- **UML (Unified Modeling Language)** is the **modeling standard**: it defines *concepts* like **class**, **attribute**, **method**, **inheritance**, **composition**, etc.
- **Mermaid** is a **text-based diagramming language**: it provides **syntax ("Mermaid code")** to *draw* a UML-style diagram.

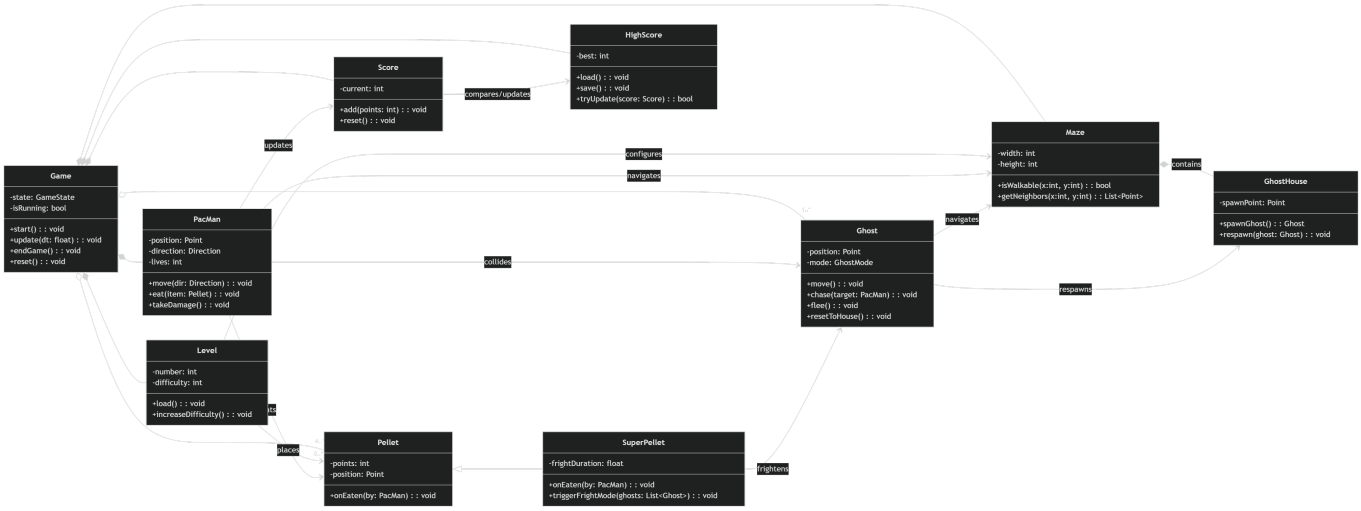
☑ Best sentence to teach:

> **"This is a UML class diagram expressed using Mermaid code."**

---

1) The Mermaid Code We're Reading

The diagram below is written in **Mermaid code**. The *meaning* we interpret is UML meaning.



Important:

- The lines like `class Game { ... }` are **Mermaid code**.
- The ideas like "Game has methods and attributes" are **UML concepts**.

---

2) How to Read a UML Class (Seen Through Mermaid Code)

2.1 UML Concept: "Class"

A **UML class** is a blueprint describing:

- **Attributes** (data / state)

- **Operations** (methods / behavior)

2.2 Mermaid Code That Represents a UML Class

In Mermaid, a UML class is written like:

2.3 What Students Should Say (UML meaning)

> "PacMan is a class. It stores a position and has a move operation."

---

3) UML Visibility (Public/Private) and the Mermaid Symbols

UML Concept: Visibility

- **Public** = accessible from other classes
- **Private** = internal to the class

Mermaid Code Symbols

- `+` means **public**
- `-` means **private**

Example from the Mermaid code:

Explain in UML language:

> "Score keeps its current value private, and exposes add() publicly."

---

4) UML Attributes vs Methods (How to Spot Them)

Attributes (state)

Mermaid format:

```
-name: Type
```

Example:

- `-lives: int`
- `-mode: GhostMode`

Methods (behavior)

Mermaid format:

```
+methodName(params): ReturnType
```

Example:

- `+eat(item: Pellet): void`
- `+isWalkable(x:int, y:int): bool`

☑ Student rule:

> **If it has parentheses `()` it's an operation (method). If not, it's an attribute.**

---

5) UML Relationships (and Their Mermaid Code)

This section is where most confusion happens. We will always do it in **two steps**:

1. Show the **Mermaid relationship syntax**
2. Translate into **UML meaning**

5.1 Inheritance (UML: Generalization)

Mermaid code

UML meaning

- **SuperPellet IS A Pellet**
- SuperPellet inherits Pellet's attributes/operations

☑ Student explanation:

> "SuperPellet is a specialized kind of Pellet that adds extra behavior."

5.2 Composition (UML: Strong "Part-of")

Mermaid code

UML meaning

- **Game owns Level and Maze**
- Their lifecycle is tightly connected to Game

☑ Student explanation:

> "Level and Maze are essential parts of Game. If Game ends, those parts don't exist as part of that game instance."

5.3 Aggregation (UML: Weak "Has-a" / Collection)

Mermaid code

UML meaning

- Game **has** a set of Ghosts and Pellets
- They can appear/disappear during play

☑ Multiplicity (how many?)

- `"1..*"` = one or more
- `"0..*"` = zero or more

☑ Student explanation:

> "A Game has at least one Ghost, and may have zero or many Pellets."

5.4 Association / Dependency (UML: "Uses")

Mermaid code

UML meaning

- PacMan **uses** Maze to move
- PacMan **uses** Score to record points

☑ Student explanation:

> "PacMan depends on Maze for movement rules and updates Score when it collects items."

---

6) Reading the Diagram Like a Story (Model Explanation)

Students should practice narrating the system in **UML terms**, while referring back to **Mermaid code** as the evidence.

Here's a model explanation:

Story Explanation (UML meaning, supported by Mermaid code)

1. **Game is the controller class**:
    - In Mermaid: `Game *-- Level`, `Game *-- Maze`, `Game *-- PacMan`, etc.
    - UML meaning: Game composes key components.
2. **Level configures the Maze and places pellets**:
    - In Mermaid: `Level --> Maze : configures` and `Level --> "0..*" Pellet : places`
    - UML meaning: Level sets up a stage layout and content.
3. **Maze contains GhostHouse**:
    - In Mermaid: `Maze *-- GhostHouse : contains`
    - UML meaning: GhostHouse is a structural part of Maze.
4. **PacMan navigates Maze, eats pellets, updates score**:

- In Mermaid: `PacMan --> Maze : navigates`, `PacMan --> Pellet : eats`, `PacMan --> Score : updates`
- UML meaning: PacMan interacts with gameplay objects and scorekeeping.

5. **Ghosts navigate Maze and respawn in GhostHouse**:
   - In Mermaid: `Ghost --> Maze : navigates`, `Ghost --> GhostHouse : respawns`
   - UML meaning: Ghost behavior depends on map and respawn area.
6. **SuperPellet is a Pellet that frightens Ghosts**:
   - In Mermaid: `Pellet <|-- SuperPellet` and `SuperPellet --> Ghost : frightens`
   - UML meaning: inheritance + interaction effect.
7. **HighScore stores best score across games**:
   - In Mermaid: `Score --> HighScore : compares/updates`
   - UML meaning: Score compares itself to HighScore and may update it.

---

7) A Student "How-To" Template (Answer Framework)

When students are asked to explain a class diagram, they can follow this script:

Step-by-step explanation template

1. **Name key classes** and state their roles.
2. For each key class, describe:
   - attributes (state)
   - operations (behavior)
3. Identify inheritance:
   - "X is a type of Y"
4. Identify composition/aggregation:
   - "X owns Y" vs "X has Y"
5. Identify associations:
   - "X uses Y to do Z"
6. Finish with a full narrative of the system.

---

8) Common Confusions (Fixed with Correct Language)

Confusion #1: "Is this UML code?"

☑ Correction:

> "UML is not code; UML is the model. The Mermaid code is the syntax used to render it."

Confusion #2: "Is `*--` a Mermaid thing or UML thing?"

☑ Correct explanation:

- `*--` is **Mermaid syntax**

- It *represents* the UML concept of **composition**

Confusion #3: "Does `-->` mean a method call?"

☑ Correction:

> "In class diagrams, arrows usually show structural relationships (uses/association), not runtime method calls."

---

9) Mini Practice Questions (With Mermaid Evidence)

1. **Which class controls the system?**
   - Look for most composition lines: `Game *-- ...`
2. **Where is inheritance used?**
   - `Pellet <|-- SuperPellet`
3. **What class stores persistent results?**
   - `HighScore` with `load()`, `save()`
4. **How do we know PacMan updates score?**
   - `PacMan --> Score : updates`

Summary

> **We interpret UML meaning (classes, attributes, relationships), and we use Mermaid code only as the written notation that draws the UML class diagram.**
